Java Problem Statements Using OOP and Exception Handling

The following problem statements are designed for students to practice Object-Oriented Programming (OOP) concepts and exception handling in Java. Each problem uses a different real-world scenario and incorporates exceptions such as `ArithmeticException`, `IndexOutOfBoundsException`, `NullPointerException`, `NumberFormatException`, `IllegalArgumentException`, and other commonly encountered exceptions.

All problems require students to use `try`, `catch`, and `finally` blocks along with classes, objects, constructors, encapsulation, and methods.

Problem 1: Banking System Using OOP and Exception Handling

Problem Statement

Consider a Banking Management System in which customers can create a bank account, deposit money, withdraw money, and display their account details. The system must handle exceptional situations, such as attempting to withdraw more money than the available balance or depositing a negative amount.

Develop a Java application using OOP concepts to demonstrate exception handling in banking operations.

Application Requirements

1. Create a class named `BankAccount` with the following private data members:

- `accountNumber` (int)
- `accountHolderName` (String)
- `balance` (double)

2. Implement a parameterized constructor to initialize the account details.

3. Define the following methods:

- `deposit(double amount)`: Deposits money after validating the amount.
- `withdraw(double amount)`: Withdraws money after checking the available balance.
- `displayAccount()`: Displays the account details.
- `getBalance()`: Returns the current balance.

4. Use a `try` block to perform deposit and withdrawal operations.

5. Use appropriate `catch` blocks to handle:

- `IllegalArgumentException` when the deposit or withdrawal amount is zero or negative.
- `ArithmeticException` when a withdrawal operation results in an invalid balance according to the application's rules.

6. Use a `finally` block to display a message indicating that the banking transaction has been processed.

7. In the `main()` method, create a `BankAccount` object, perform a deposit or withdrawal operation, handle exceptions, and display the final account details.

Expected Behaviour

Suppose an account has a balance of ₹10,000, and the customer attempts to withdraw ₹15,000.

- The withdrawal operation detects insufficient funds and throws an exception.
- The `catch` block displays an appropriate error message.
- The `finally` block displays the transaction completion message.
- The account balance remains ₹10,000.

Concepts Demonstrated: Encapsulation, constructors, objects, method validation, `IllegalArgumentException`, custom exception handling, and `finally`.

Note: Students may implement insufficient funds using a custom exception or `IllegalArgumentException`. `ArithmeticException` should be used only if the program deliberately performs an invalid arithmetic operation; insufficient funds do not automatically generate it.

Problem 2: Student Marks and Result Management System

Problem Statement

Consider a Student Result Management System in which a teacher can enter student marks, calculate the average marks, and display the student's result. The system must handle exceptional situations, such as dividing by zero when no subjects are registered or entering marks in an invalid format.

Develop a Java application using OOP concepts to demonstrate exception handling during result processing.

Application Requirements

1. Create a class named `Student` with the following private data members:

- `rollNumber` (int)
- `studentName` (String)
- `marks` (double array)
- `numberOfSubjects` (int)

2. Implement a parameterized constructor to initialize the student details.

3. Define the following methods:

- `addMarks(int subjectIndex, double mark)`: Stores marks for a subject after validation.
- `calculateAverage()`: Calculates and returns the average marks.
- `displayResult()`: Displays the student details, marks, average, and result.

4. Use a `try` block to enter marks and calculate the average.

5. Use appropriate `catch` blocks to handle:

- `ArithmeticException` when the application explicitly performs an invalid arithmetic operation, such as division by zero.
- `IndexOutOfBoundsException` when a subject index is outside the valid range.
- `NumberFormatException` when marks entered as text cannot be converted into a numeric value.
- `IllegalArgumentException` when marks are below 0 or above 100.

6. Use a `finally` block to display a message indicating that result processing has been completed.

7. In the `main()` method, create a `Student` object, enter marks, calculate the average, handle exceptions, and display the final result.

Expected Behaviour

Suppose a student has registered for three subjects, but the teacher attempts to enter marks at index 5.

- The application attempts to store the marks.
- The `catch` block handles the `IndexOutOfBoundsException`.
- The `finally` block displays the completion message.
- The application displays the valid marks already stored.

Concepts Demonstrated: Encapsulation, arrays, constructors, validation, `ArithmeticException`, `IndexOutOfBoundsException`, and `NumberFormatException`.

Problem 3: Online Shopping Cart Management System

Problem Statement

Consider an Online Shopping Cart in which customers can add products, remove products, calculate the total bill, and display cart details. The system must handle exceptional situations, such as attempting to access a product that does not exist in the cart, adding a product with invalid details, or performing operations on an uninitialized cart.

Develop a Java application using OOP concepts to demonstrate exception handling in shopping cart operations.

Application Requirements

1. Create a class named `CartItem` with the following private data members:

- `productId` (int)
- `productName` (String)
- `price` (double)
- `quantity` (int)

2. Implement a parameterized constructor to initialize product details.

3. Define the following methods:

- `calculateSubtotal()`: Returns the product of price and quantity.
- `updateQuantity(int newQuantity)`: Updates the quantity after validation.
- `displayItem()`: Displays the product details.

4. Create a class named `ShoppingCart` with a private array of `CartItem` objects.

5. Define methods to add items, remove items, calculate the total bill, and display the cart.

6. Use a `try` block to perform cart operations.

7. Use appropriate `catch` blocks to handle:

- `IndexOutOfBoundsException` when accessing an invalid cart index.
- `NullPointerException` when attempting to access a cart item that has not been initialized.
- `IllegalArgumentException` when the price is negative or the quantity is zero or negative.
- `ArithmeticException` if the application explicitly performs an invalid arithmetic operation.

8. Use a `finally` block to display the cart operation completion message.

9. In the `main()` method, create a shopping cart, add items, attempt an invalid operation, and display the final cart contents.

Expected Behaviour

Suppose the cart contains a laptop and a mouse. The customer attempts to access an item at index 5 in a cart whose capacity is 3.

- The application attempts to access the requested item.
- The `catch` block handles the `IndexOutOfBoundsException`.
- The `finally` block displays the completion message.
- The valid cart items remain available for display.

Concepts Demonstrated: Composition, arrays of objects, encapsulation, constructors, `NullPointerException`, and `IndexOutOfBoundsException`.

Problem 4: Employee Salary and Payroll Management System

Problem Statement

Consider an Employee Payroll Management System in which an administrator can create employee records, calculate salaries, apply deductions, and display salary details. The system must handle exceptional situations, such as entering salary details in an invalid format or attempting to calculate salary using invalid working days.

Develop a Java application using OOP concepts to demonstrate exception handling in payroll processing.

Application Requirements

1. Create a class named `Employee` with the following private data members:

- `employeeId` (int)
- `employeeName` (String)
- `basicSalary` (double)
- `workingDays` (int)

2. Implement a parameterized constructor to initialize the employee details.

3. Define the following methods:

- `calculateSalary()`: Calculates the payable salary according to the working days.
- `applyDeduction(double deduction)`: Deducts an amount after validating it.
- `displaySalarySlip()`: Displays employee and salary details.
- `getNetSalary()`: Returns the final salary.

4. Use a `try` block to create employee records and calculate salaries.

5. Use appropriate `catch` blocks to handle:

- `NumberFormatException` when salary or working days entered as text cannot be converted into a numeric value.
- `IllegalArgumentException` when salary or working days are invalid.
- `ArithmeticException` when an explicitly coded invalid arithmetic operation occurs.

6. Use a `finally` block to display a message indicating that payroll processing has been completed.

7. In the `main()` method, create an `Employee` object, calculate salary, apply deductions, handle exceptions, and display the final salary slip.

Expected Behaviour

Suppose an employee's basic salary is ₹30,000. The administrator enters `"thirty thousand"` instead of a numeric salary.

- The application attempts to convert the input into a number.
- The `catch` block handles the `NumberFormatException`.
- The `finally` block displays the payroll processing completion message.
- The application does not update the salary using the invalid input.

Concepts Demonstrated: Encapsulation, constructors, numeric conversion, validation, `NumberFormatException`, and `finally`.

Problem 5: Library Book Issuing and Returning System

Problem Statement

Consider a Library Management System in which a librarian can add books, issue books to students, return books, and display book details. The system must handle exceptional situations, such as attempting to issue an unavailable book, accessing an invalid book position, or performing an operation on a book object that has not been initialized.

Develop a Java application using OOP concepts to demonstrate exception handling in library operations.

Application Requirements

1. Create a class named `Book` with the following private data members:

- `bookId` (int)
- `bookTitle` (String)
- `authorName` (String)
- `isIssued` (boolean)

2. Implement a parameterized constructor to initialize the book details.

3. Define the following methods:

- `issueBook()`: Issues the book if it is available.
- `returnBook()`: Marks the book as returned.
- `displayBook()`: Displays the book details.
- `isBookIssued()`: Returns the current issue status.

4. Create a class named `Library` containing an array of `Book` objects.

5. Define methods to add books, issue books by index, return books by index, and display all books.

6. Use a `try` block to perform library operations.

7. Use appropriate `catch` blocks to handle:

- `IndexOutOfBoundsException` when the requested book index is invalid.
- `NullPointerException` when a book position contains no initialized object.
- `IllegalStateException` when attempting to issue a book that is already issued or return a book that has not been issued.

8. Use a `finally` block to display a message indicating that the library operation has been completed.

9. In the `main()` method, create a library, add books, attempt to issue or return a book, handle exceptions, and display the final library records.

Expected Behaviour

Suppose a book has already been issued to a student. The librarian attempts to issue the same book again.

- The `issueBook()` method detects that the book is already issued.
- The method throws an `IllegalStateException`.
- The `catch` block displays an appropriate error message.
- The `finally` block displays the completion message.
- The book's issue status remains unchanged.

Concepts Demonstrated: Object composition, arrays of objects, encapsulation, `IllegalStateException`, `NullPointerException`, and `IndexOutOfBoundsException`.

Problem 6: Hospital Patient Appointment Management System

Problem Statement

Consider a Hospital Appointment Management System in which hospital staff can register patients, schedule appointments, update appointment details, and display patient information. The system must handle exceptional situations, such as attempting to access an unregistered patient, entering a patient age in an invalid format, or providing missing patient information.

Develop a Java application using OOP concepts to demonstrate exception handling in hospital appointment management.

Application Requirements

1. Create a class named `Patient` with the following private data members:

- `patientId` (int)
- `patientName` (String)
- `age` (int)
- `doctorName` (String)

2. Implement a parameterized constructor to initialize the patient details.

3. Define the following methods:

- `scheduleAppointment(String doctorName)`: Assigns a doctor after validation.
- `updatePatientAge(int newAge)`: Updates the patient's age after validation.
- `displayPatient()`: Displays the patient details.
- `getPatientName()`: Returns the patient's name.

4. Use a `try` block to register patients and schedule appointments.

5. Use appropriate `catch` blocks to handle:

- `NumberFormatException` when the patient's age entered as text cannot be converted into an integer.
- `NullPointerException` when the application attempts to use a patient reference that is `null`.
- `IllegalArgumentException` when the age is negative or the doctor name is blank.
- `IllegalStateException` when an appointment cannot be scheduled because the patient record is incomplete.

6. Use a `finally` block to display a message indicating that appointment processing has been completed.

7. In the `main()` method, create a patient object, schedule an appointment, handle exceptions, and display the final patient information.

Expected Behaviour

Suppose the application receives the patient age as `"twenty-five"`.

- The application attempts to convert the input into an integer.
- The conversion fails and generates a `NumberFormatException`.
- The `catch` block displays an appropriate error message.
- The `finally` block displays the completion message.
- The patient record is not updated with an invalid age.

Concepts Demonstrated: Encapsulation, object creation, validation, `NumberFormatException`, `NullPointerException`, and `IllegalArgumentException`.

Problem 7: Railway Reservation Management System

Problem Statement

Consider a Railway Reservation System in which passengers can book seats, cancel reservations, and display ticket details. The system must handle exceptional situations, such as attempting to book a seat that does not exist, cancelling a seat that has not been booked, or entering the passenger count in an invalid format.

Develop a Java application using OOP concepts to demonstrate exception handling during railway reservation operations.

Application Requirements

1. Create a class named `Reservation` with the following private data members:

- `trainNumber` (int)
- `trainName` (String)
- `passengerName` (String)
- `seatNumber` (int)
- `isBooked` (boolean)

2. Implement a parameterized constructor to initialize the reservation details.

3. Define the following methods:

- `bookSeat(int seatNumber)`: Books an available seat.
- `cancelReservation()`: Cancels an existing reservation.
- `displayTicket()`: Displays the reservation details.
- `isReservationActive()`: Returns the booking status.

4. Create a class named `Train` containing an array of `Reservation` objects or a seat-status array.

5. Define methods to book seats, cancel reservations, and display available seats.

6. Use a `try` block to perform booking and cancellation operations.

7. Use appropriate `catch` blocks to handle:

- `IndexOutOfBoundsException` when the requested seat index is outside the available range.
- `IllegalArgumentException` when the seat number is zero or negative.
- `IllegalStateException` when attempting to cancel a reservation that does not exist.
- `NumberFormatException` when a seat number entered as text cannot be converted into an integer.

8. Use a `finally` block to display a message indicating that reservation processing has been completed.

9. In the `main()` method, create a train reservation object, attempt a booking or cancellation, handle exceptions, and display the final reservation status.

Expected Behaviour

Suppose a train has 50 seats, but a passenger attempts to book seat number 75.

- The application validates the seat number.
- The booking operation throws an `IndexOutOfBoundsException` or a suitable validation exception.
- The `catch` block displays an appropriate error message.
- The `finally` block displays the completion message.
- No invalid seat is booked.

Concepts Demonstrated: Encapsulation, arrays, object composition, validation, `IndexOutOfBoundsException`, and `IllegalStateException`.

Problem 8: Hotel Room Booking and Billing System

Problem Statement

Consider a Hotel Management System in which staff can allocate rooms, calculate customer bills, and display booking details. The system must handle exceptional situations, such as attempting to allocate a room that is unavailable, calculating a bill with an invalid number of days, or converting an invalid room rate into a numeric value.

Develop a Java application using OOP concepts to demonstrate exception handling in hotel operations.

Application Requirements

1. Create a class named `HotelRoom` with the following private data members:

- `roomNumber` (int)
- `roomType` (String)
- `pricePerNight` (double)
- `isBooked` (boolean)

2. Implement a parameterized constructor to initialize the room details.

3. Define the following methods:

- `bookRoom()`: Books the room if it is available.
- `calculateBill(int numberOfDays)`: Calculates the total room charge.
- `releaseRoom()`: Marks the room as available.
- `displayRoomDetails()`: Displays the room details.

4. Use a `try` block to perform room booking and billing operations.

5. Use appropriate `catch` blocks to handle:

- `IllegalStateException` when attempting to book an already booked room.
- `IllegalArgumentException` when the room rate is negative or the number of days is zero or negative.
- `NumberFormatException` when the room rate entered as text cannot be converted into a numeric value.
- `ArithmeticException` if the application explicitly performs an invalid arithmetic operation.

6. Use a `finally` block to display a message indicating that the hotel operation has been completed.

7. In the `main()` method, create a `HotelRoom` object, book a room, calculate the bill, handle exceptions, and display the final room details.

Expected Behaviour

Suppose a hotel room costs ₹3,000 per night. A customer enters `-2` as the number of booking days.

- The billing method validates the number of days.
- The method throws an `IllegalArgumentException`.
- The `catch` block displays an appropriate error message.
- The `finally` block displays the completion message.
- No invalid bill is generated.

Concepts Demonstrated: Encapsulation, constructors, method validation, `IllegalStateException`, `IllegalArgumentException`, and `finally`.

Problem 9: Inventory Stock Management System

Problem Statement

Consider an Inventory Management System in which a store manager can add stock, sell products, update stock quantities, and display inventory details. The system must handle exceptional situations, such as selling more units than are available, attempting to access an invalid product index, or entering the stock quantity in an invalid format.

Develop a Java application using OOP concepts to demonstrate exception handling in inventory operations.

Application Requirements

1. Create a class named `InventoryItem` with the following private data members:

- `itemId` (int)
- `itemName` (String)
- `stockQuantity` (int)
- `price` (double)

2. Implement a parameterized constructor to initialize the item details.

3. Define the following methods:

- `addStock(int quantity)`: Adds stock after validating the quantity.
- `sellStock(int quantity)`: Reduces stock if sufficient stock is available.
- `calculateInventoryValue()`: Returns the total value of the available stock.
- `displayInventory()`: Displays item details and current stock.

4. Create a class named `Inventory` containing an array of `InventoryItem` objects.

5. Define methods to add inventory items, update stock, sell items by index, and display all inventory records.

6. Use a `try` block to perform stock management operations.

7. Use appropriate `catch` blocks to handle:

- `IndexOutOfBoundsException` when accessing an invalid inventory index.
- `IllegalArgumentException` when stock quantity or price is invalid.
- `NumberFormatException` when stock quantity entered as text cannot be converted into an integer.
- `IllegalStateException` when the requested stock cannot be sold because insufficient stock is available.

8. Use a `finally` block to display a message indicating that inventory processing has been completed.

9. In the `main()` method, create an inventory object, add stock, attempt a sale, handle exceptions, and display the final inventory details.

Expected Behaviour

Suppose a store has 20 units of a product. A customer attempts to purchase 25 units.

- The `sellStock()` method checks the available quantity.
- The method throws an `IllegalStateException` because the requested quantity exceeds the available stock.
- The `catch` block displays an appropriate error message.
- The `finally` block displays the completion message.
- The stock quantity remains 20.

Concepts Demonstrated: Encapsulation, arrays of objects, constructors, validation, `IllegalStateException`, and `IndexOutOfBoundsException`.

Problem 10: Employee Attendance and Leave Management System

Problem Statement

Consider an Employee Attendance and Leave Management System in which employees can mark attendance, apply for leave, and view their attendance records. The system must handle exceptional situations, such as accessing an invalid attendance day, performing an operation on an uninitialized employee record, or entering attendance data in an invalid format.

Develop a Java application using OOP concepts to demonstrate exception handling in attendance and leave management.

Application Requirements

1. Create a class named `EmployeeAttendance` with the following private data members:

- `employeeId` (int)
- `employeeName` (String)
- `attendance` (boolean array)
- `totalWorkingDays` (int)

2. Implement a parameterized constructor to initialize the employee details and attendance array.

3. Define the following methods:

- `markAttendance(int day, boolean present)`: Updates attendance for a specified day.
- `calculateAttendancePercentage()`: Calculates the employee's attendance percentage.
- `displayAttendance()`: Displays the employee's attendance details.
- `getPresentDays()`: Returns the total number of days present.

4. Use a `try` block to mark attendance and calculate the attendance percentage.

5. Use appropriate `catch` blocks to handle:

- `IndexOutOfBoundsException` when the day number is outside the valid attendance range.
- `NullPointerException` when the attendance array or employee reference has not been initialized.
- `NumberFormatException` when the day number entered as text cannot be converted into an integer.
- `IllegalArgumentException` when the number of working days is zero or negative.

6. Use a `finally` block to display a message indicating that attendance processing has been completed.

7. In the `main()` method, create an `EmployeeAttendance` object, mark attendance, calculate the attendance percentage, handle exceptions, and display the final attendance record.

Expected Behaviour

Suppose the attendance array stores records for 30 days. The administrator attempts to mark attendance for day 35.

- The application attempts to access the specified day.
- The `catch` block handles the `IndexOutOfBoundsException`.
- The `finally` block displays the completion message.
- The existing attendance records remain unchanged.

Concepts Demonstrated: Encapsulation, arrays, constructors, `NullPointerException`, `IndexOutOfBoundsException`, and exception handling.

Summary of Exceptions Covered

Problem	Scenario	Main exceptions
1. Banking	Invalid transactions	`IllegalArgumentException`, custom exception
2. Student Result	Invalid subject index or marks	`ArithmeticException`, `IndexOutOfBoundsException`, `NumberFormatException`
3. Shopping Cart	Invalid cart item access	`IndexOutOfBoundsException`, `NullPointerException`, `IllegalArgumentException`
4. Payroll	Invalid salary input	`NumberFormatException`, `IllegalArgumentException`
5. Library	Invalid book operations	`IllegalStateException`, `NullPointerException`, `IndexOutOfBoundsException`
6. Hospital	Invalid patient details	`NumberFormatException`, `NullPointerException`, `IllegalArgumentException`
7. Railway	Invalid seat reservation	`IndexOutOfBoundsException`, `IllegalStateException`, `NumberFormatException`
8. Hotel	Invalid booking or billing	`IllegalStateException`, `IllegalArgumentException`
9. Inventory	Insufficient stock	`IllegalStateException`, `IndexOutOfBoundsException`, `NumberFormatException`
10. Attendance	Invalid attendance day	`IndexOutOfBoundsException`, `NullPointerException`, `NumberFormatException`

Instructions for Students

For each problem statement, students should:

1. Implement all required classes with private data members and appropriate constructors.
2. Use methods to validate input and control access to object data.
3. Demonstrate at least one successful operation and one operation that generates an exception.
4. Use separate `catch` blocks for the specified exception types wherever appropriate.
5. Include a `finally` block that executes whether the operation succeeds or fails.
6. Display the final object state to verify that a failed operation has not incorrectly modified the data.